

Achieving a Single Page Application (SPA) feel with Datastar involves moving away from traditional full-page reloads and instead leveraging client-side reactive signals alongside Server-Sent Events (SSE) HTML morphing. [Talk Python](#)

Instead of fetching entire pages, the browser remains on a single root page shell. Datastar dynamically patches the DOM, shifts URL paths, and binds UI state locally.

[templ.guide](#)

1. Instant Navigation & URL Syncing (SPA Routing)

To navigate without reloads, combine Datastar's GET actions with browser history actions and `data-on:click` handlers.

HTML



```
<!-- Main Navbar -->
<nav>
  <button
    data-on:click="@get('/dashboard', {contentType: 'form'})"
    data-on:click__window="history.pushState({}, '', '/dashboard')">
    Dashboard
  </button>
  <button
    data-on:click="@get('/settings', {contentType: 'form'})"
    data-on:click__window="history.pushState({}, '', '/settings')">
    Settings
  </button>
</nav>

<!-- SPA View Container -->
<main id="app-view">
  <!-- Dynamic server content morphs here -->
</main>
```

- **How it works:** The `@get` action triggers an SSE connection to the backend. The server streams back HTML fragments targeting `#app-view`, while `history.pushState` updates the address bar instantaneously without a page load.

2. Local State Management with Signals

Like React or Vue, Datastar supports fine-grained, reactive client state via signals. You don't need server round-trips for purely client-side UI states like opening dropdowns, toggling modals, or switching tabs.

HTML



```
<!-- Local Component State -->
<div data-signals="{ isModalOpen: false, selectedTab: 'overview' }">

  <!-- Toggle Modal without hitting the server -->
  <button data-on:click="$isModalOpen = true">Open Dialog</button>

  <!-- Modal Backdrop -->
  <div data-show="$isModalOpen" class="modal">
    <h3>Local SPA Modal</h3>
    <button data-on:click="$isModalOpen = false">Close</button>
  </div>

  <!-- Local Tab Switching -->
  <button data-on:click="$selectedTab = 'overview'">Overview</button>
  <button data-on:click="$selectedTab = 'analytics'">Analytics</button>

  <div data-show="$selectedTab === 'overview'">Overview Content</div>
  <div data-show="$selectedTab === 'analytics'">Analytics Content</div>
</div>
```

3. Seamless Page Transitions

SPAs feel fast because element updates animate smoothly. Datastar supports native browser View Transitions out of the box using event modifiers.

HTML



```
<!-- Smooth cross-fade DOM updates -->
<button data-on:click__viewtransition="@get('/api/load-feed')">
```

```
Refresh Feed  
</button>
```

Datastar automatically wraps DOM fragment updates inside `document.startViewTransition()`, morphing UI changes with CSS animations.

4. Real-Time Streaming (SSE Backend Patching)

For interactive experiences like chat rooms, streaming LLM text responses, live dashboards, or multi-user canvas tools, the Go backend pushes updates directly into client signals or DOM elements over Server-Sent Events.

Go Backend (Streaming Updates)

Go



```
func handleLiveStats(w http.ResponseWriter, r *http.Request) {  
    sse := datastar.NewSSE(w, r)  
  
    for count := 1; count <= 5; count++ {  
        time.Sleep(1 * time.Second)  
  
        // Push an updated HTML fragment over the SSE stream  
        sse.MergeFragment(  
            fmt.Sprintf('<div id="live-count">Count: %d</div>', count),  
        )  
    }  
}
```

Frontend Target

HTML



```
<div id="live-count">Count: 0</div>  
<button data-on:click="@post('/api/start-live-stats')">Start Stream</button>
```

5. Preserving Focus & UI State (DOM Morphing)

Standard HTMX replacements often break focus or drop user inputs when swapping `innerHTML`. Datastar uses idiomatic DOM morphing by default: [Jeff Hui + 1](#)

- If a user is typing in a search bar while an SSE stream updates the results container, input focus, scroll offsets, and cursor positions remain intact.
- You can explicitly tell Datastar to ignore elements during server updates using `data-ignore` or `data-preserve-attr`.

HTML



```
<!-- Input state and focus are kept intact when the surrounding DOM updates -  
<input type="text" data-bind="searchQuery" data-preserve-attr="value" />
```

Summary Architecture: The Datastar "SPA Pattern"

Feature	Traditional SPA (React/ Vue)	Datastar + Go Approach
State	Redux / Pinia / JS Memory	<code>data-signals</code> (client) + Server State
Routing	React Router / Client bundle	<code>history.pushState</code> + SSE Fragment Fetching
Rendering	Client VDOM	Server-Side Templates (<code>templ</code>) + Idiomatic Morphing
Real-time	Custom WebSockets	Native Datastar SSE streams